Notebook de Programação Competitiva

Índice

- Grafos
- Matemática
- Strings
- Estruturas de Dados

Tabela ASCII (32–126)

```
Dec Hex Char | Dec Hex Char | Dec Hex Char | Dec Hex Char
-----------------+-------------------+-------------------+----------------
32 20 (esp) | 48 30 0 | 64 40 @ | 80 50 P
33 21 ! | 49 31 1 | 65 41 A | 81 51 Q
34 22 " | 50 32 2 | 66 42 B | 82 52 R
35 23 # | 51 33 3 | 67 43 C | 83 53 S
36 24 $ | 52 34 4 | 68 44 D | 84 54 T
37 25 % | 53 35 5 | 69 45 E | 85 55 U
38 26 & | 54 36 6 | 70 46 F | 86 56 V
39 27 ' | 55 37 7 | 71 47 G | 87 57 W
40 28 ( | 56 38 8 | 72 48 H | 88 58 X
41 29 ) | 57 39 9 | 73 49 I | 89 59 Y
42 2A * | 58 3A : | 74 4A J | 90 5A Z
43 2B + | 59 3B ; | 75 4B K | 91 5B [
44 2C , | 60 3C < | 76 4C L | 92 5C
45 2D - | 61 3D = | 77 4D M | 93 5D ]
46 2E . | 62 3E > | 78 4E N | 94 5E ^
47 2F / | 63 3F ? | 79 4F O | 95 5F _
-----------------+-------------------+-------------------+----------------
96 60 ` | 112 70 p | 120 78 x |
97 61 a | 113 71 q | 121 79 y |
98 62 b | 114 72 r | 122 7A z |
99 63 c | 115 73 s | 123 7B { |
100 64 d | 116 74 t | 124 7C | |
101 65 e | 117 75 u | 125 7D } |
102 66 f | 118 76 v | 126 7E ~ |
103 67 g | 119 77 w | |
104 68 h | | |
105 69 i | | |
106 6A j | | |
107 6B k | | |
108 6C l | | |
109 6D m | | |
110 6E n | | |
111 6F o | | |
```

Grafos

BFS

Busca em largura em **O(V+E)**.

```
// BFS - O(V + E)
vector<vector<int>> adj;   // lista de adjacência

void bfs(int start) {
    queue<int> q;
    vector<bool> visited(GRAPH_MAX_SIZE, false);

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        // Processa o vértice u aqui, se necessário

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
```

DFS

Busca em Profundidade em **O(V+E)**

```
// DFS - O(V + E)
vector<vector<int>> adj;   // lista de adjacência
vector<bool> visited;

void dfs(int u) {
    visited[u] = true;

    // Processa o vértice u aqui, se necessário

    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v);
        }
    }
}
```

UNION FIND

```
#include <bits/stdc++.h>

using namespace std;

const int N_MAX = 100001;
int link[N_MAX];
int size[N_MAX];

// Função para encontrar o representante do conjunto
int find(int x) {
    return (x == link[x]) ? x : (link[x] = find(link[x]));
}

// Função para verificar se dois elementos estão no mesmo conjunto
bool same(int x, int y) {
    return find(x) == find(y);
}

// Função para unir dois conjuntos
void unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return;
    if (size[a] < size[b]) swap(a, b);
    size[a] += size[b];
    link[b] = a;
}
//exemplo na main
int main() {

    int N, K, i, x1, x2;
    char op;

    cin >> N >> K;

    // Inicialização da estrutura DSU
    for (i = 0; i < N; ++i) {
        link[i] = i;
        size[i] = 1;
    }

    for (i = 0; i < K; ++i) {
        cin >> op >> x1 >> x2;
        if (op == 'C') {
            if (same(x1, x2)) {
                cout << "S" << '\n';
            } else {
                cout << "N" << '\n';
            }
```

```
        } else { // 'U'
            if(!same(x1, x2))unite(x1, x2);
        }
    }

    return 0;
}
```

DIJKSTRA

Menor caminho com pesos positivos em **O((V+E) log V)**.

```
// DIJKSTRA - O((V + E) log V)
// A lista de adjacência 'adj' deve armazenar pares {peso, vertice}.
vector<int> dijkstra(int s, int n, const vector<vector<pair<int, int>>>& adj) {
    const int INF = 1e9; // Usar um valor grande como infinito
    vector<int> dist(n, INF);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int,
int>>> pq;

    dist[s] = 0;
    pq.push({0, s}); // Fila de prioridade armazena {distancia, vertice}

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d > dist[u]) {
            continue; // Já encontramos um caminho mais curto para 'u'
        }

        for (auto [w, v] : adj[u]) { // Para cada vizinho 'v' de 'u' com peso 'w'
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

EDMONDS-KARP

Calcula o fluxo máximo em um grafo. A complexidade é **O(V * E²)**.

```
// Edmonds-Karp - O(V * E^2)
const int MAX = 110; // Número máximo de vértices
const long long INF = 1e18;
```

```
int n; // Número de vértices
long long capacity[MAX][MAX];
vector<int> adj[MAX];

// Encontra um caminho de aumento usando BFS
long long bfs(int s, int t, vector<int>& parent) {
    fill(parent.begin(), parent.end(), -1);
    parent[s] = -2;
    queue<pair<int, long long>> q;
    q.push({s, INF});

    while (!q.empty()) {
        int u = q.front().first;
        long long flow = q.front().second;
        q.pop();

        for (int v : adj[u]) {
            if (parent[v] == -1 && capacity[u][v] > 0) {
                parent[v] = u;
                long long new_flow = min(flow, capacity[u][v]);
                if (v == t) {
                    return new_flow;
                }
                q.push({v, new_flow});
            }
        }
    }
    return 0; // Nenhum caminho de aumento encontrado
}

// Função principal do Edmonds-Karp
long long edmonds_karp(int s, int t) {
    long long max_flow = 0;
    vector<int> parent(n + 1);
    long long new_flow;

    // Enquanto houver um caminho de aumento
    while ((new_flow = bfs(s, t, parent)) > 0) {
        max_flow += new_flow;
        int current = t;
        while (current != s) {
            int prev = parent[current];
            // Atualiza capacidades no grafo residual
            capacity[prev][current] -= new_flow;
            capacity[current][prev] += new_flow;
            current = prev;
        }
    }
    return max_flow;
}

// Para usar:
// 1. Defina 'n' (número de vértices).
// 2. Preencha a matriz 'capacity' e a lista 'adj'.
```

```
//    Para arestas bidirecionais, adicione capacidade em ambos os sentidos.
// 3. Chame edmonds_karp(source, sink).
// 4. Lembre-se de limpar as estruturas (capacity, adj) para múltiplos casos de
teste.
```

Matemática

FAST POW

Exponenciação Rápida (Binária) Calcula (a^b)%m de forma eficiente. A complexidade é **O(logb)**.

Esta é a versão modular, ideal para problemas que pedem o resultado sob um módulo (como 10^9 +7) para evitar que os números fiquem grandes demais (overflow). Se um problema pedir apenas o cálculo de a^b e o resultado couber em um long long, basta adaptar a função removendo o parâmetro m e todas as operações de módulo (%).

```
// Exponenciação Rápida (Binária) - O(log b)
// Versão para cálculo modular: (a^b) % m
long long binpow(long long a, long long b, long long m) {
    // A linha abaixo é importante caso 'a' seja maior que 'm'.
    // Se o problema pedir um módulo, é sempre bom garantir
    // que os números manipulados permaneçam pequenos.
    a %= m;

    long long res = 1;
    while (b > 0) {
        // Se o bit menos significativo de b for 1 (ou seja, b é ímpar)
        if (b & 1)
            // Multiplica o resultado por 'a' e aplica o módulo
            // para manter o número dentro do limite.
            res = (res * a) % m;

        // Eleva 'a' ao quadrado e aplica o módulo.
        // Isso evita que 'a' cresça excessivamente.
        a = (a * a) % m;

        b >>= 1; // b = b / 2
    }
    return res;
}
```